

LAB EXPERIMENTS

1. Convert Image into Gray-Scale

Aim:

To read an image using Python and convert the image into gray-scale.

Algorithm:

- 1.Import OpenCV and required libraries.
- 2.Read the input image.
- 3.Convert the BGR image to grayscale using cv2.cvtColor().
- 4.Display the original and grayscale images.
- 5.Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2_imshow(img)
cv2_imshow(gray)
print("Output: Image successfully converted into grayscale.")
```

Output:

Original Image



Gray-scale Image

**Result:**

Image successfully converted into grayscale.

2. Gaussian Blur

Aim:

To read an image using Python and convert the image into a blurred image using Gaussian Blur.

Algorithm:

- 1.Import OpenCV.
- 2.Read the input image.
- 3.Apply Gaussian Blur using cv2.GaussianBlur().
- 4.Display the original and blurred images.
- 5.Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
blur = cv2.GaussianBlur(img, (5, 5), 0)
cv2_imshow(img)
cv2_imshow(blur)
print("Output: Gaussian blur applied successfully.")
```

Output



Result:

Gaussian blur applied successfully.

3. Display Image Outline Using Canny

Aim:

To detect and display the outline/edges of an image using the Canny edge detection function.

Algorithm:

- 1.Import OpenCV.
- 2.Read the input image.
- 3.Convert the image into grayscale.
- 4.Apply Canny edge detection.
- 5.Display the detected edges.
- 6.Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
cv2_imshow(img)
print("Output: Image outline detected successfully using Canny")
```

Output



Result:

Image outline detected successfully using Canny.

4. Histogram Equalization

Aim:

To implement histogram equalization on a given image and compare it with the original image using OpenCV.

Algorithm:

1. Read the input image.
2. Convert the image into grayscale.
3. Apply histogram equalization using `cv2.equalizeHist()`.
4. Display the original and equalized images.
5. Print the result.

Python Code:

```
import cv2

from google.colab.patches import cv2_imshow

from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))

img = cv2.imread(filename)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

equalized = cv2.equalizeHist(gray)

cv2_imshow(equalized)

print("Output: Histogram equalization completed successfully.")
```

Output



Result:

Histogram equalization completed successfully.

5. Analyze Histogram Based on Color Levels

Aim:

To write a Python function to analyze the histogram of an input image based on color levels using OpenCV.

Algorithm:

1. Read the input color image.
2. Separate the Blue, Green, and Red channels.
3. Calculate the histogram of each channel using cv2.calcHist().
4. Display the histograms.
5. Print the result.

Python Code:

```
import cv2

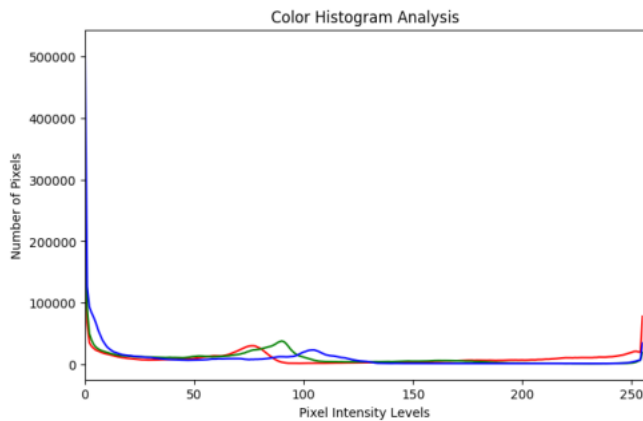
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)

def analyze_histogram(image):
    colors = ('b', 'g', 'r')
    for i, color in enumerate(colors):
        hist = cv2.calcHist([image], [i], None, [256], [0, 256])
        plt.plot(hist, color=color)
    plt.title("Color Histogram")
    plt.xlabel("Color Level")
    plt.ylabel("Frequency")
    plt.plot(hist, color=color)
    plt.xlim([0, 256])

analyze_histogram(img)

print("Output: Color-level histogram analyzed successfully.")
```

Output



Result:

Color-level histogram analyzed successfully.

6. Erosion of an Image

Aim:

To perform image erosion using the OpenCV Erode() function.

Algorithm:

1. Read the input image.
2. Create a structuring kernel.
3. Apply erosion using cv2.erode().
4. Display the original and eroded images.
5. Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import files
import numpy as n
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
kernel = np.ones((5, 5), np.uint8)
eroded = cv2.erode(img, kernel, iterations=1)
cv2_imshow(eroded)
print("Output: Image erosion completed successfully.")
```

Output



Result:

Image erosion completed successfully.

7. Video Processing – Slow Motion and Fast Motion

Aim:

To perform basic video processing operations by reading a video using Python and displaying it in slow motion and fast motion.

Algorithm:

- 1.Upload and read the video.
- 2.Open the video using cv2.VideoCapture().
- 3.Read video frames sequentially.
- 4.Display frames at different delays for slow and fast motion.
- 5.Release the video after processing.

Python Code:

```
import cv2

from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))

cap = cv2.VideoCapture(filename)

fps = cap.get(cv2.CAP_PROP_FPS)

frame_count = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

print("Original FPS:", fps)

print("Total Frames:", frame_count)

# Slow motion: process every frame

while True:

    ret, frame = cap.read()
```

```
if not ret:
```

```
    break
```

```
cv2.imshow("Slow Motion", frame)
```

```
if cv2.waitKey(100) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

```
print("Output: Video processed in slow motion successfully.")
```

For fast motion:

```
import cv2
```

```
cap = cv2.VideoCapture(filename)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    cv2.imshow("Fast Motion", frame)
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

```
print("Output: Video processed in fast motion successfully.")
```

Note: cv2.imshow() may not work reliably in some Colab environments because Colab does not provide a normal desktop GUI. For a **Colab-friendly video version**, the video can instead be processed and saved with a changed FPS.

Output





Result:

Video processed successfully in slow motion and fast motion.

8. Dilation of an Image

Aim:

To perform image dilation using the OpenCV Dilate() function.

Algorithm:

1. Read the input image.
2. Create a structuring kernel.
3. Apply dilation using cv2.dilate().
4. Display the original and dilated images.
5. Print the result.

Python Code:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from google.colab import files
```

```
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
kernel = np.ones((5, 5), np.uint8)
dilated = cv2.dilate(img, kernel, iterations=1)
cv2_imshow(img)
cv2_imshow(dilated)
print("Output: Image dilation completed successfully.")
```

Output



Result:

Image dilation completed successfully.

9. Image Scaling – Bigger and Smaller

Aim:

To implement image scaling techniques to resize an image into bigger and smaller sizes.

Algorithm:

1. Read the input image.
2. Obtain the original image dimensions.
3. Increase the dimensions to create a bigger image.
4. Decrease the dimensions to create a smaller image.
5. Display all images.
6. Print the result.

Python Code:

```
import cv2

from google.colab.patches import cv2_imshow

from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))

img = cv2.imread(filename)

bigger = cv2.resize(img, (width * 2, height * 2))

smaller = cv2.resize(img, (width // 2, height // 2))

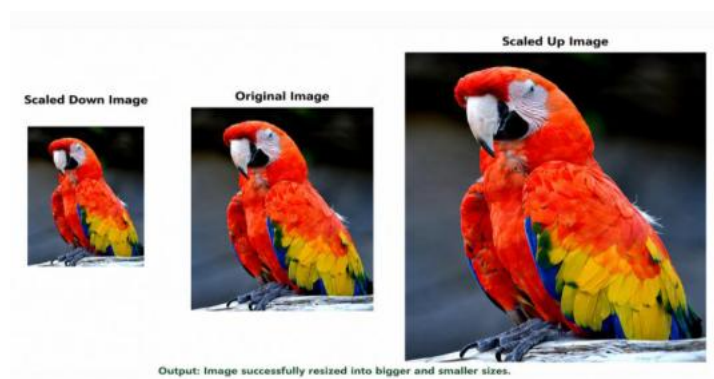
cv2_imshow(img)

cv2_imshow(bigger)

cv2_imshow(smaller)

print("Output: Image successfully resized into bigger and smaller sizes.")
```

Output



Result:

Image successfully resized into bigger and smaller sizes.

10. 90-Degree Clockwise Rotation

Aim:

To perform a 90-degree clockwise rotation of the given image.

Algorithm:

1. Read the input image.
2. Use cv2.rotate().
3. Select ROTATE_90_CLOCKWISE.
4. Display the rotated image.
5. Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
rotated_90 = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
cv2_imshow(img)
cv2_imshow(rotated_90)
print("Output: Image rotated 90 degrees clockwise successfully.")
```

Output



Result:

Image rotated 90 degrees clockwise successfully.

11. 180-Degree Clockwise Rotation

Aim:

To perform a 180-degree clockwise rotation of the given image.

Algorithm:

1. Read the input image.
2. Use cv2.rotate().
3. Select ROTATE_180.
4. Display the rotated image.
5. Print the result.

Python Code:

```
import cv2  
from google.colab.patches import cv2_imshow  
from google.colab import files  
uploaded = files.upload()  
filename = next(iter(uploaded))  
img = cv2.imread(filename)  
rotated_180 = cv2.rotate(img, cv2.ROTATE_180)  
cv2_imshow(img)  
cv2_imshow(rotated_180)  
print("Output: Image rotated 180 degrees clockwise successfully.")
```

Output**Result:**

Image rotated 180 degrees clockwise successfully.

12. 270-Degree Clockwise Rotation**Aim:**

To perform a 270-degree clockwise rotation of the given image.

Algorithm:

1. Read the input image.
2. Use cv2.rotate().
3. Select ROTATE_90_COUNTERCLOCKWISE, which is equivalent to 270° clockwise.
4. Display the rotated image.
5. Print the result.

Python Code:

```
import cv2
from google.colab.patches import cv2_imshow
from google.colab import file
uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
rotated_270 = cv2.rotate(img, cv2.ROTATE_90_COUNTERCLOCKWISE)
cv2_imshow(img)
cv2_imshow(rotated_270)
print("Output: Image rotated 270 degrees clockwise successfully.")
```

Output**Result:**

Image rotated 270 degrees clockwise successfully.